P R O J E C T R E V I E W D O C U M E N T

Review 2 Report

Range-Aware Edge AI Quantization Compiler

A Compiler Design Course Project

Course: BCSE307L — Compiler Design

Programme: B.Tech. Computer Science and Engineering

Vellore Institute of Technology, Vellore

Submitted by:

Yash Pradhan [24BCE0702]

Anjini Pandey [24BCE0714]

Ishita Srivastava [24BDS0234]

Faculty Guide: Prof. Kanagaraj R

September 2026

Introduction

The **Range-Aware Edge AI Quantization Compiler** is a Compiler Design course project that examines how static analysis can guide a precision-reducing optimization. It uses a deliberately small C-like numerical language to make the compiler stages and their safety decisions visible; it is not intended to be a full C compiler or a production machine-learning framework.

The motivating setting is edge inference, where memory capacity, compute throughput, energy, and data movement can be constrained. A signed INT8 value occupies one byte, compared with four bytes for an FP32 value, so INT8 can reduce storage requirements and may support efficient integer arithmetic on suitable hardware. However, replacing FP32 with INT8 indiscriminately is unsafe: a value may exceed the representable interval [-128, 127], may be fractional, or may change the result of downstream computation.

The project therefore asks whether a compiler can establish, before execution, which values are safe to represent using INT8. After the source program is parsed and checked, the compiler constructs Three Address Code (TAC), a control-flow graph (CFG), and Static Single Assignment (SSA) form. Interval-based range analysis is combined with an integrality property; the safety pass accepts only transformations justified by those facts. If safety cannot be proven, the original representation is retained.

The intended outcome is an explainable, machine-independent compiler pipeline with source-level traceability and a lightweight simulator/backend for checking the behaviour of original and transformed IR within the defined language. The report records the current implementation and functional verification separately from performance claims: runtime, latency, energy, and hardware-power improvements are not asserted without measurements. Section 1 presents the problem and motivation in detail, while Section 4 describes the architecture and implementation.

1. Problem Statement and Motivation

1.1 Background

Machine learning inference is increasingly being pushed away from centralized cloud servers and onto resource-constrained edge devices such as microcontrollers, mobile processors, and embedded accelerators. Zhou et al. survey this shift under the term edge intelligence and argue that running inference close to the data source reduces latency, limits bandwidth consumption, and improves data privacy compared to cloud-only deployment. Chen and Ran conduct a complementary review of how deep learning workloads are adapted to run within these constraints and observe that the combination of increasingly capable edge hardware and increasingly efficient model and system design has made on-device inference practical for a growing range of applications.

Edge devices, however, differ from server-class hardware along axes that matter directly to numerical computation: available memory is limited, compute units may lack dedicated floating-point hardware or may execute floating-point instructions far more slowly than integer instructions, and battery-powered deployments make every additional instruction and every additional byte of memory traffic a measurable cost. A program compiled with the assumption of abundant 32-bit floating-point (FP32) computation may therefore be functionally correct but practically difficult to deploy on such hardware.

Reducing the numerical precision used to represent intermediate values is one of the most direct ways to address these constraints. Representing a value using an 8-bit integer (INT8) instead of a 32-bit float reduces its storage footprint by a factor of four and, on hardware with integer-optimized arithmetic units, can reduce execution time and energy consumption. This is the motivation behind quantization schemes explored extensively in the machine learning systems literature. This project treats the same underlying idea, replacing FP32 computation with INT8 computation wherever this can be shown to be safe, as a problem to be solved inside a compiler, rather than as a property established empirically for a trained neural network model.

1.2 Problem Statement

The central question this project addresses is:

> *Can a compiler statically determine, before a program is ever executed, whether a given numerical computation can be safely represented using 8-bit integers, and transform only those computations for which this safety can be formally established?*

This framing separates two things that are often conflated in applied quantization work: the decision of whether a transformation is safe, and the transformation itself. Naively replacing every FP32 variable in a program with an INT8 variable is unsafe in general. It can silently introduce:

- overflow, when a computed value falls outside the representable range [-128, 127];
- incorrect representation, when a value is not an integer to begin with, so that truncation or rounding changes its meaning;
- loss of exactness, when arithmetic that is exact in FP32 no longer is once intermediate results are confined to an 8-bit range; and
- incorrect downstream computation, since a single unsafe conversion can propagate through every subsequent operation consuming the corrupted value.

This project therefore defines its problem as a compiler-based, safety-aware optimization problem: given a source program, identify, through static analysis alone, without executing the program, the subset of numerical operations that can probably be represented in INT8 without any of the failures

above, and transform only that subset. Every value for which safety cannot be proven statically is left in its original FP32 representation.

1.3 Motivation

The motivation for the project follows a short chain of reasoning:

- Edge devices operate under real resource constraints — memory, compute throughput, and energy.
- These constraints create a need for compact, efficient numerical representations.
- Reduced-precision formats such as INT8 are an attractive way to meet this need.
- Converting FP32 computation to INT8 without justification risks overflow and incorrect results.
- A dependable reduction in precision therefore requires reasoning, at compile time, about what values a computation can take.
- Static range analysis, combined with an integrality check, is a well-studied way to perform this reasoning.
- Only once this reasoning has certified a computation as safe should the compiler perform the precision-reducing transformation.

Each of these steps corresponds to a concrete stage in the compiler pipeline proposed in Section 4.

1.4 Why a Compiler-Based Solution?

Framing the safety question as a compiler problem, rather than as a runtime check or a property established empirically by testing a trained model, has concrete advantages. A compiler already constructs an intermediate representation (IR) of the program in which every value has an explicit definition and a well-defined set of uses; this structure is exactly what is required to track how a numerical property, such as range or integrality, propagates through the program. Compilers are also the natural setting for data-flow analysis: propagating facts along a control-flow graph, merging information at join points, and iterating to a fixed point over loops are well-understood compiler-construction techniques. A compiler-based decision is also machine-independent in the sense used in the Compiler Design syllabus — the analysis depends only on the numeric semantics of the source language, not on a specific target processor — so the resulting safety proof is a property of the program rather than of one particular execution. Finally, deciding safety at compile time avoids the overhead of runtime range checks on a device that is already resource constrained.

1.5 Relevance to Compiler Design

The project is deliberately scoped so that its technical content sits inside standard Compiler Design topics rather than inside machine learning: Three Address Code as an intermediate representation, basic-block and control-flow-graph construction, static single assignment form, data-flow analysis, and a machine-independent optimization pass that consumes the results of that analysis to perform a code transformation. Edge AI motivates why the optimization is useful; it does not change the technical content of the optimization itself. The project is not an exercise in training or evaluating a neural network, and it does not depend on any deep learning framework.

2. Objectives of the Project

The project is guided by the following objectives:

- Design and formally specify a small C-like numerical language — supporting integer and floating-point constants, variable declarations, assignment, arithmetic and relational operators, if/else, while, and print — that is sufficient to express computations whose safety for INT8 representation is worth analyzing.
- Implement a lexer and a parser that tokenize and parse this language into an abstract syntax tree, attaching source line and column information to every token and AST node.
- Perform semantic analysis on the AST (declaration checking and operand-type consistency) and translate validated programs into Three Address Code, correctly handling expressions, assignments, and control flow through labels and backpatching.
- Construct basic blocks and a control-flow graph from the generated TAC, correctly identifying leaders and handling both conditional branching and loop back-edges.
- Translate the control-flow graph into static single assignment form, inserting phi nodes at dominance-frontier join points using a standard construction algorithm.
- Design and implement a static range and integrality analysis over the SSA representation, including interval-based transfer rules for each supported operator, a conservative merge rule at phi nodes, and a bounded widening strategy that guarantees termination on programs containing while loops.
- Define and implement a conservative set of INT8 safety rules that combine the results of range and integrality analysis, and implement a quantization transformation pass that rewrites only the operations proven safe, leaving every other value unchanged.
- Preserve source-level traceability throughout the pipeline so that every quantization decision can be reported back to its originating line and column, and evaluate the resulting system on representative test programs using well-defined, measured (not assumed) metrics.

These objectives are scoped to be realistic for a single-semester student project: each depends only on the compiler stage that precedes it, and none requires infrastructure — such as a full LLVM backend, a deep learning framework, or hardware-specific code generation — beyond what is described in Section 4.

3. Literature Survey / Related Work

This survey draws on original, verifiable sources spanning edge computing, neural network quantization, compiler-based optimization, and static/data-flow analysis, retrieved from conference proceedings, journal archives, and arXiv preprints. It is organized to move from the practical motivation for the project (edge constraints and quantization) toward the theoretical and systems foundations the proposed compiler pipeline is built on (static analysis, SSA, and compiler infrastructure).

3.1 Edge AI and Efficient Computation

Edge intelligence — the deployment of AI inference directly on edge devices rather than in centralized data centers — has been surveyed extensively as network and device capabilities have matured. Zhou et al. review the architectures, frameworks, and enabling technologies for running deep learning models at the network edge, and identify constrained memory, constrained compute, and constrained energy budgets as the three recurring bottlenecks separating edge deployment from cloud deployment. Chen and Ran conduct a complementary review focused on how deep learning workloads are partitioned,

compressed, and adapted to run within these constraints, covering model compression, hardware acceleration, and edge-cloud collaboration as the main mitigation strategies used in practice. Both surveys converge on an observation directly relevant to this project: whatever technique is used, it must reduce the resource cost of computation without requiring specialized training infrastructure at the edge itself. Reducing numerical precision is one of the few techniques that directly reduces memory footprint and, on suitable hardware, computation cost, without altering the structure of the program.

3.2 Quantization and Reduced-Precision Computation

Quantization, in the machine learning systems literature, generally refers to representing the weights and activations of a neural network using a lower-precision numeric format than the 32-bit floating-point representation used during training. Jacob et al. describe a quantization scheme and a co-designed training procedure allowing inference to be carried out using integer-only arithmetic on mobile CPUs, reporting close to a fourfold reduction in memory footprint from moving activations and weights to 8-bit integers. Krishnamoorthi provides a broader overview of quantization techniques for convolutional networks, including post-training quantization and quantization-aware training, and recommends per-channel weight quantization combined with per-layer activation quantization as a practical default for INT8 deployment. Both works treat quantization primarily as a numerical technique applied to an already-trained model, whose safety is established empirically by measuring the resulting change in model accuracy, rather than proved for each individual value. This project adopts the target representation used in this literature — signed 8-bit integers in the range [-128, 127] — but asks a different question: rather than whether a whole model tolerates quantization on average, whether an individual computation, considered in isolation, can be proved safe to quantize before the program is ever executed. This reframes quantization as a compiler-level, formally checked transformation rather than a model-level, statistically validated one.

3.3 Quantization Safety and Numerical Constraints

The quantization literature is well aware that reducing precision is not a simple datatype substitution. Practical INT8 schemes such as those of Jacob et al. and Krishnamoorthi rely on a scale factor, and in some schemes a zero-point, that map a continuous range of real values onto the 256 representable 8-bit integers, together with calibration procedures chosen to keep quantization error acceptably small across a representative dataset. These techniques operate at the level of tensors of weights and activations, and their correctness is judged statistically, in terms of end-to-end model accuracy after quantization. The proposed project intentionally works with a narrower notion of safety, inherited from the static-analysis literature rather than from the quantization literature: a value is converted to INT8 only if it can be shown, through static range and integrality analysis, to always lie inside [-128, 127] and to always be an integer, on every possible execution path. This is a stricter and more limited guarantee than statistical calibration provides — the project does not attempt per-channel scaling, zero-point calibration, or accuracy-based tuning, which are explicitly left outside its scope — but it is a guarantee that holds exactly, rather than on average, for every value to which it is applied.

3.4 Compiler-Based and Hardware-Aware Optimization

Several existing compiler systems demonstrate that machine-independent, IR-based analysis and transformation can be extended to numerically and hardware-sensitive optimizations. Lattner and Adve’s LLVM defines a low-level, strongly typed intermediate representation held in SSA form specifically so that analyses and transformations can be applied uniformly across compile time, link time, and run time, independent of source language and target machine. Ragan-Kelley et al.'s Halide separates the algorithmic description of an image-processing pipeline from its execution schedule,

letting a compiler search over parallelization, tiling, and memory-locality choices for a given hardware target without changing program semantics. Chen et al.'s TVM extends this idea to deep-learning workloads, exposing graph-level and operator-level optimizations, including hardware-specific code generation, through a compiler stack rather than hand-written kernels for each backend. These systems share a structural idea this project borrows directly: numerical and hardware-driven optimizations are implemented as compiler passes over an explicit IR, guided by static analysis, rather than as ad hoc source-level transformations. The proposed project is far smaller in scope than LLVM, Halide, or TVM — it targets one specific transformation, FP32 -to-INT8 conversion under proven safety, for one small language — but follows the same architectural principle of separating IR construction, analysis, and transformation into distinct, inspectable compiler stages.

3.5 Static Analysis, Data-Flow Analysis and Range Analysis

The theoretical basis for the proposed range and integrality analysis comes from the abstract interpretation framework introduced by Cousot and Cousot. Their 1976 paper on static determination of dynamic properties of programs introduces interval-based abstraction — representing the possible values of a variable at a program point by an interval [minimum, maximum] — as a computable, sound approximation of a program's exact runtime behaviour. Their 1977 POPL paper generalizes this into abstract interpretation as a unified lattice-theoretic framework, showing that a wide class of static analyses can be expressed as the construction or approximation of a fixed point of a monotone function over a lattice of abstract program properties. A well-known practical difficulty with this framework, directly relevant to the loop-analysis component of this project, is that some abstract lattices — including interval lattices — have infinite height, so a naive fixed-point iteration over a loop may not terminate; Bourdoncle addresses this by formalizing widening and narrowing operators and giving efficient iteration strategies, based on a weak topological ordering of the program's control-flow graph, that guarantee termination while bounding the resulting loss of precision. Blanchet et al. describe Astrée, an interval - and abstract-interpretation-based static analyzer built to prove the absence of run-time errors, including arithmetic overflow, in large safety-critical embedded C programs, reporting that it can do so with very few or no false alarms on real avionics code. Astrée is a large-scale demonstration that the class of static reasoning proposed here — bounding the range of a variable and using that bound to rule out a specific class of run-time error — is both theoretically sound and practically deployable on embedded software, which is precisely the target domain motivating this project's edge-AI framing.

3.6 SSA and Compiler Optimization

Static single assignment form was popularized as a practical intermediate representation by Cytron, Ferrante, Rosen, Wegman, and Zadeck, who give an efficient algorithm, based on dominance frontiers, for inserting the phi functions needed to give every variable definition a single, unambiguous static location. Because every SSA variable has exactly one definition, a data-flow fact computed for that variable — such as an interval and an integrality flag — can be associated directly with its single defining instruction, rather than with a definition that may vary depending on which control-flow path was taken to reach a use. This property is what makes SSA convenient for the proposed range and integrality analysis: the analysis for a given SSA variable needs only to inspect the operation that defines it and, at a phi node, needs only to merge the facts already computed for its incoming operands. LLVM's use of SSA as its core representation reflects the same motivation at industrial scale. The proposed pipeline builds SSA directly from the CFG using the classical dominance-frontier construction before running range analysis, so the analysis itself can be described, and implemented, purely in terms of per-instruction transfer functions and a join operation at phi nodes.

3.7 Research / Implementation Gap

None of the systems reviewed above targets exactly the problem this project addresses, but it would be inaccurate to claim that no existing work touches on any part of it — quantization systems reason about numerical precision; Astrée and the abstract-interpretation literature reason about static value ranges; and LLVM, Halide, and TVM demonstrate compiler infrastructures that perform hardware-and precision-aware transformations. The gap this project identifies is one of scope and pedagogical transparency rather than raw novelty. Industrial quantization pipelines operate on trained models and validate safety statistically over a dataset; industrial static analyzers such as Astrée are large, highly tuned systems built to scale to hundreds of thousands of lines of avionics C code; and general-purpose compiler infrastructures such as LLVM expose numerical optimizations as a small number of passes within a much larger system. This project instead proposes a compact pipeline in which SSA construction, range and integrality analysis, conservative safety decision-making, and precision transformation are each implemented as separate, individually inspectable compiler stages over a small language, with the explicit goal of making the underlying compiler-optimization principles easy to study, trace, and defend in a Compiler Design course setting, rather than of matching the scale or coverage of the systems discussed above.

3.8 Literature Comparison Table

Author(s) / Year	Work / System	Problem Addressed	Key Technique	Relevant Concept	Difference from Proposed Project
Zhou et al., 2019 [1]	Edge Intelligence survey	Motivating constraints for on-device AI	Survey / taxonomy of edge-AI architectures	Edge resource constraints	Provides motivation only; no compiler or static-analysis content
Chen & Ran, 2019 [2]	Deep Learning with Edge Computing: A Review	Adapting DL workloads to edge hardware	Survey of compression / offloading / acceleration	Edge deployment constraints	Model/system-level review; no compile-time safety proofs
Jacob et al., 2018 [3]	Quantization for integer-only inference	Efficient integer-only NN inference	Quantization-aware training + integer scheme	INT8 representation	Model-level, statistically validated; no per-value static proof
Krishnamoorthi, 2018 [4]	Quantizing CNNs (whitepaper)	Practical CNN quantization guidance	Per-channel / per-layer quantization, calibration	Scale / zero-point quantization	Empirical, accuracy-based safety; no compiler IR or static analysis
Cousot & Cousot, 1976 [5]	Static determination of dynamic properties	Approximating runtime value behaviour statically	Interval abstraction of variable values	Range-analysis foundation	General theory; project instantiates it for INT8 safety
Cousot & Cousot, 1977 [6]	Abstract interpretation (POPL)	Unifying static analyses as fixpoint computation	Lattice-based abstract interpretation	Fixed-point static analysis	General framework; project applies a narrow interval + integrality instance
Bourdoncle, 1993 [7]	Chaotic iteration with widenings	Guaranteeing termination over infinite-height lattices	Widening / narrowing, weak topological order	Loop convergence strategy	Reused at a small, teaching scale for while-loop termination

Author(s) / Year	Work / System	Problem Addressed	Key Technique	Relevant Concept	Difference from Proposed Project
Blanchet et al., 2003 [8]	Astrée static analyzer	Proving absence of run-time errors in embedded C	Abstract-interpretation interval/numeric domains	Industrial-scale range analysis	Full production analyzer; project is a compact analogue for a small language
Cytron et al., 1991 [9]	SSA construction (TOPLAS)	Efficiently building SSA form	Dominance-frontier-based phi insertion	SSA / phi nodes	Foundational algorithm; applied directly to build the project's IR
Lattner & Adve, 2004 [10]	LLVM	Lifelong, language-independent analysis/transformation	SSA-based typed IR + pass infrastructure	SSA IR, machine-independent optimization	Full production compiler infrastructure; project is single-purpose and far smaller
Ragan-Kelley et al., 2013 [11]	Halide	Optimizing image pipelines for parallelism/locality	Algorithm/ schedule separation, compiler search	Machine-independent, hardware-aware optimization	Targets scheduling, not numeric-precision safety
Chen et al., 2018 [12]	TVM	End-to-end optimizing compiler for DL across hardware	Graph + operator-level compiler optimization	Hardware-aware compiler optimization	Full DL compiler stack; project isolates just the precision-safety sub-problem

3.9 Literature Synthesis

Read together, the literature surveyed above traces a clear path toward the proposed project. Edge intelligence research establishes that on-device inference is constrained by memory, compute, and energy in ways that make reduced-precision computation attractive. The quantization literature shows that INT8 representation is a practical, widely adopted way to realize that reduction, while also showing that quantization decisions are normally made empirically, at the level of a whole trained model. The abstract-interpretation and static-analysis literature supplies the theoretical machinery — interval abstraction, fixed-point computation, and widening for loop termination — needed to make a safety decision for an individual value rather than for a whole model, and Astrée demonstrates that this machinery scales to real embedded software when built carefully. SSA, as formalized by Cytron et al. and adopted at industrial scale by LLVM, provides the intermediate representation on which such an analysis is naturally expressed. Finally, systems such as Halide and TVM confirm that treating numerical and hardware-aware optimization as an explicit compiler pass, separate from the source program, is both a sound and a practically useful architectural choice. The proposed Range-Aware Edge AI Quantization Compiler combines these ideas — SSA, static range and integrality analysis, and a conservative INT8 safety check — into a single, compact pipeline, built at a scale appropriate for a Compiler Design course project rather than for production deployment, while remaining faithful to the compiler-construction techniques on which it is built.

4. Proposed Methodology (Architecture / Design)

This section describes the planned compiler pipeline in enough detail to show how each stage will actually be built, how it connects to the stages before and after it, and how the design principles stated in Section 1 (conservative safety, guaranteed loop termination, and full source traceability) are enforced concretely at each stage.

Sections 4.1–4.15 retain the Review 1 design baseline as originally submitted. Sections 4.16–4.21 report the verified Review 2 implementation.

4.1 Overall System Architecture

The compiler is organized as a linear pipeline of well-defined stages, shown in Figure 1. Each stage consumes the representation produced by the previous stage and produces a new representation, so that every stage can be implemented, tested, and reasoned about independently.

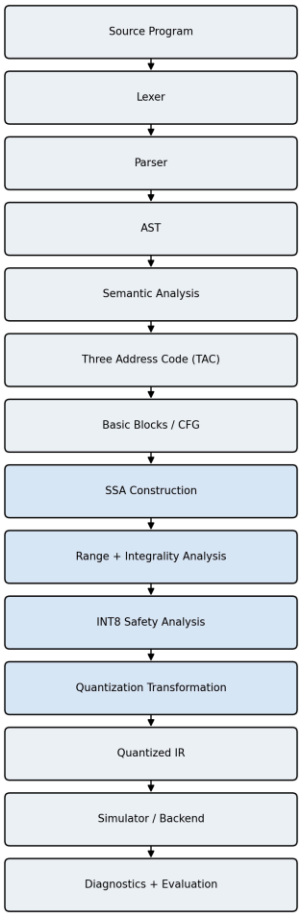


Figure 1. Overall compiler pipeline, from source program to diagnostics.

4.1.1 Module Descriptions

Figure 1 presents the end-to-end workflow. The following descriptions clarify each stage's purpose, input, output, and role without replacing the architecture diagram or the existing module-responsibility summary.

1. Lexer. Input: source characters. Output: tokens carrying line and column positions. It recognizes identifiers, numeric literals, operators, and keywords so later stages can parse the program and trace diagnostics back to the source.

2. Parser and Abstract Syntax Tree (AST). Input: token stream. Output: an AST whose nodes retain source locations. The parser enforces the mini-language grammar and gives later semantic and translation stages a structured representation of expressions and statements.

3. Semantic Analysis. Input: AST. Output: a validated program representation or source-located diagnostics. It checks declaration/use rules and operand-type consistency before code generation, preventing malformed or semantically invalid constructs from silently entering the IR.

4. Three Address Code (TAC) Generation. Input: semantically validated AST. Output: TAC with explicit temporaries, labels, and branch instructions. This decomposition makes intermediate operations individually visible to the later range and integrality analysis.

5. Basic Blocks and Control Flow Graph (CFG). Input: TAC. Output: basic blocks connected by control-flow edges. Leader-based block formation identifies branch targets and fall-through boundaries; loop back-edges and conditional branches expose the flow required by SSA and analysis.

6. Static Single Assignment (SSA) Construction. Input: CFG and TAC definitions. Output: versioned SSA values with phi nodes at required control-flow joins. Each SSA value has one definition, making facts easier to associate with the operation that produces the value.

7. Range and Integrality Analysis. Input: SSA and CFG. Output: an interval [minimum, maximum] and an integrality fact for each analyzable value. Arithmetic transfer rules propagate facts, phi nodes merge incoming facts, and conservative loop widening/fallback prevents unbounded analysis.

8. INT8 Safety Analysis. Input: inferred range, integrality, and operation information. Output: a per-value safety decision and rejection reason where applicable. A transformation is eligible only when the bounded range fits [-128, 127], integrality is guaranteed, and the operation is supported; unknown or unsafe cases are rejected conservatively.

9. Selective Quantization Transformation. Input: SSA plus safety decisions. Output: quantized SSA for the proven-safe subset. Eligible operations are rewritten to the project's INT8 representation, while rejected or unproven values retain their original FP32 form.

10. Lightweight Simulator / Backend. Input: original and transformed IR. Output: execution results for the supported mini-language programs. It enables functional comparison within the project scope; it is not a production backend or a measurement of physical edge-device performance.

11. Diagnostics and Evaluation. Input: source-location metadata, analysis results, transformation decisions, and test programs. Output: reports showing ranges, integrality, pass/reject status, and reasons linked to source lines and columns. Evaluation includes automated tests and representative examples; runtime, latency, energy, and hardware-power improvements remain unclaimed unless measured.

Stage	Responsibility
Lexer	Convert source characters into a token stream, attaching line/column position to every token.
Parser	Build an abstract syntax tree from the token stream according to the mini-language grammar.
Semantic Analysis	Check declarations and operand-type consistency; validate control-flow constructs.
TAC Generation	Lower the AST into Three Address Code, including labels and branches for control flow.
Basic Blocks / CFG	Partition TAC into basic blocks and connect them into a control-flow graph.
SSA Construction	Rename variable definitions and insert phi nodes at dominance-frontier join points.
Range + Integrality Analysis	Propagate interval and integrality facts over the CFG in SSA form; widen to guarantee loop termination.
INT8 Safety Analysis	Apply the conservative safety rule to every SSA value using its computed range and integrality.
Quantization Transformation	Rewrite only the operations proven safe into INT8-typed instructions; leave the rest unchanged.
Simulator / Backend	Execute both the original and the quantized IR and compare results for correctness.
Diagnostics / Evaluation	Report, per source location, the range, integrality, and safety decision for each value.

4.2 Frontend Design

The frontend of the compiler turns source text into a validated abstract syntax tree without losing track of where each construct came from in the original file. The lexer scans the source character stream and emits a token for each lexeme — identifiers, numeric literals (distinguishing integer from floating-point literals), operators, and keywords — attaching to every token the line and column at which it began. The parser consumes this token stream and builds an AST according to the grammar of the mini language described in Section 2; each AST node stores the source position of the token, or of the first token of the sub-expression, from which it was built. Semantic analysis walks the completed AST to check that every variable is declared before use, that operand types are consistent for each operator, and that if, while, and print constructs are well formed. Any semantic error is reported using its recorded line and column.

4.3 Intermediate Representation

Once a program has passed semantic analysis, it is lowered into Three Address Code, in which every instruction computes at most one operator applied to at most two operands, for example:

```
t1 = 10
t2 = 20
t3 = t1 + t2
```

Conditional and unconditional control flow is expressed using labels and jump instructions, for example:

```
if t1 goto L1
goto L2
L1: ...
L2: ...
```

TAC is a convenient intermediate representation for this project because it exposes every intermediate value as an explicitly named temporary — exactly the granularity at which range and integrality analysis needs to operate. A source expression such as `(a + b) * c` cannot be usefully range-analyzed as a single unit, but its TAC decomposition into explicit temporaries can be.

4.4 Basic Blocks and CFG

Basic blocks are formed from the TAC instruction stream using the standard leader-based algorithm: the first instruction, the target of any jump, and the instruction immediately following any jump or conditional jump are each marked as leaders, and each basic block consists of a leader together with all instructions up to (but not including) the next leader. Edges are added between blocks according to the control flow implied by jumps and fall-through execution; a conditional branch produces two outgoing edges, and a while loop produces a back edge from the end of the loop body to the loop-condition block, which is what makes the resulting CFG cyclic. Figure 2 shows the basic-block structure produced for a simple if/else statement. This CFG is the structure over which SSA construction and range analysis are subsequently performed; without an explicit basic-block/CFG stage there is no well-defined notion of a join point at which two differently valued definitions of the same variable must be merged.

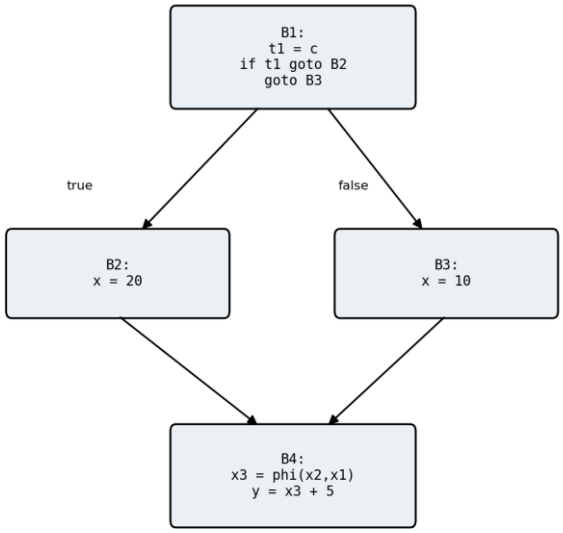


Figure 2. Basic blocks and control-flow graph for an if/else statement.

4.5 SSA Construction

SSA construction proceeds in two conceptual steps over the CFG built in the previous stage: first, dominance frontiers are computed for every basic block, following the definition used by Cytron et al.; second, a phi function is inserted for a variable at every block in the iterated dominance frontier of a block containing a definition of that variable, after which every variable definition and use is renamed with a fresh version number. For the example program in Figure 3 (left), this produces the SSA form shown on the right.

Original program	**SSA form**

`x = 10`
`if (c)`
`  x = 20`
`y = x + 5`
	`x1 = 10`
	`if (c)`
	`  x2 = 20`
	`x3 = phi(x1, x2)`
	`y1 = x3 + 5`

Figure 3. Original program and its static single assignment form.

This representation is useful for the proposed range analysis specifically because every SSA name has exactly one static definition point: the analysis can therefore be expressed purely as a per-instruction transfer function (given the ranges of my operands, what is my range) plus a merge rule at phi nodes, without needing to reason about which of several possible definitions of a variable is live at a given program point.

4.6 Range and Integrality Analysis

Every SSA value is abstracted, following the interval-analysis tradition established by Cousot and Cousot [5], as a pair consisting of an interval [minimum, maximum] and a boolean integrality flag. Representative transfer rules for the arithmetic operators supported by the mini language are given below.

```
Addition:        [a,b] + [c,d]  ->  [a+c, b+d]
Subtraction:     [a,b] - [c,d]  ->  [a-d, b-c]
Multiplication:  [a,b] * [c,d]  ->  [min(ac,ad,bc,bd), max(ac,ad,bc,bd)]
Division:        if 0 in [c,d]  ->  UNKNOWN (conservative)
                 else computed analogously via reciprocal endpoints
```

Multiplication considers all four endpoint products because the sign of the operands determines which combination produces the extreme values; division is treated conservatively as UNKNOWN whenever the divisor interval contains zero, since a value that may be divided by zero cannot be given a sound finite interval. Integrality is tracked alongside the interval: a value is integral if both its operands are integral and the operator preserves integrality — this holds for addition, subtraction, and multiplication over integers, but not, in general, for division. At a control-flow join, realized as a phi node in SSA form, the ranges of the incoming operands are merged conservatively by taking the union of their intervals and the logical AND of their integrality flags, since a value that is only sometimes integral must be treated as non-integral for a safety proof that must hold on every execution path.

4.7 Phi Nodes and Data-Flow Propagation

Because SSA already localizes the effect of control-flow merging into phi nodes, the propagation rule at a join point is simple to state and implement. Given x1 → [10,10] (integral) and x2 → [20,20] (integral), with `x3 = phi(x1, x2)`, the analysis computes x3 → [10,20] (integral), as shown in Figure 4. This merge is intentionally conservative — it does not attempt to track which branch was actually taken,

since that is generally undecidable statically — which is consistent with the overall design principle that the analysis must never claim a tighter range than a program can actually exhibit.

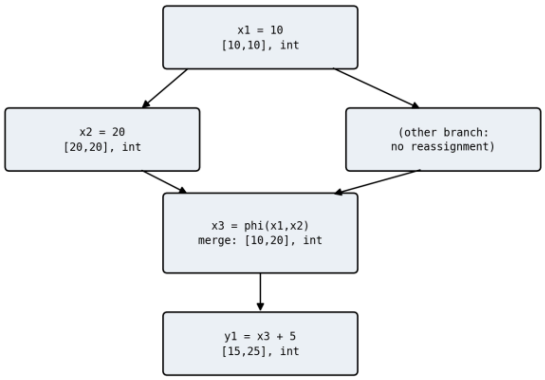


Figure 4. Range propagation through a phi node and a subsequent addition.

4.8 Loop Analysis and Convergence

While loops introduce a back edge in the CFG and, correspondingly, a phi node for every loop-carried variable at the loop header, for example:

```
i = 0
while (i < 100)
    i = i + 1
```

produces a phi node merging the initial value of i with the value of i at the end of the loop body. Because this phi node's own output is one of its own inputs, a naive fixed-point iteration that simply re-evaluates the transfer functions can fail to terminate over an interval lattice of infinite height, a well-documented difficulty in the abstract-interpretation literature. The proposed analysis addresses this using a widening strategy in the style of Bourdoncle: after a bounded number of iterations around a loop header without stabilizing, the algorithm applies a widening step that extrapolates a still-growing interval bound toward infinity, treated in this project's INT8 setting as UNKNOWN rather than as a literal unbounded interval, instead of continuing to iterate indefinitely. This guarantees termination: the analysis performs at most a small, bounded number of ordinary iterations per loop header before either stabilizing on a fixed range or falling back conservatively to UNKNOWN. In either case, the design principle stated throughout this document is enforced — if a stable, safe bound cannot be established for a loop-carried variable, that variable is simply never proposed for INT8 quantization.

4.9 INT8 Safety Rules

A value is judged SAFE for INT8 representation only if all of the following hold:

- its range is known (not UNKNOWN);
- the minimum of its range is at least -128;
- the maximum of its range is at most 127;
- it is guaranteed integral;
- it was produced only by operations the quantization pass supports; and
- no other safety violation (such as a division whose denominator range contains zero) was flagged for it.

```
x -> [10,20],   integral = true   =>  SAFE
x -> [120,200], integral = true   =>  REJECT (possible overflow: 200 > 127)
x -> [1.5,1.5], integral = false  =>  REJECT (non-integral)
x -> UNKNOWN                      =>  REJECT (safety cannot be proven)
```

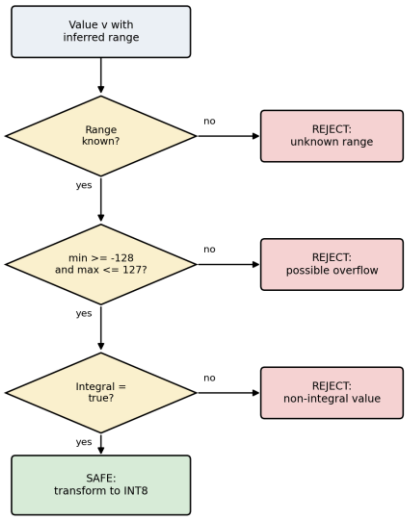


Figure 5. INT8 safety decision procedure.

This rule is deliberately conservative: every value it labels SAFE is probably safe, at the cost of also rejecting some values that would, in fact, be safe but whose safety the analysis is not precise enough to establish — for example, a loop-carried variable whose range was widened to UNKNOWN even though its true range happens to be bounded. This is an accepted and expected trade-off in static analysis: soundness is preserved even where precision is not.

4.10 Quantization Transformation

The quantization pass is kept strictly separate from the analysis that decides safety: the analysis only labels values, and the transformation pass only acts on values already labelled SAFE. For example, given

```
t1 = 10.0
t2 = 20.0
t3 = t1 + t2
```

if t1, t2, and t3 are all found SAFE, the pass rewrites this to

```
t1 : int8 = 10
t2 : int8 = 20
t3 : int8 = t1 + t2
```

retyping the operands and the operation. Any value not labelled SAFE — including any value that participates in the same expression as an unsafe value, since an INT8 operation cannot be silently mixed with an FP32 operand — is left in its original FP32 form. This separation of concerns (decide, then transform) keeps the safety analysis independently testable: it can be checked against hand-worked examples without the transformation pass existing yet, and the transformation pass can be checked by verifying that it only ever acts on values already labelled SAFE.

4.11 Source-Level Diagnostics

Because line and column information is attached to every token in the frontend and carried forward through the AST, TAC, and SSA representations, every quantization decision can be reported back to the exact source location responsible for it, for example:

```
Line 14, Column 7
Variable: x
Inferred range: [120, 200]
Integral: YES
INT8 Safety: REJECTED
Reason: possible overflow
```

This kind of diagnostic serves two purposes: it makes every optimization decision auditable, so a reviewer can check the compiler's reasoning against the source program rather than having to trust an opaque pass; and, more practically, it makes the analysis and transformation passes far easier to test and debug during implementation, since an incorrect safety decision can be traced directly back to the line of source code that produced it.

4.12 Backend / Execution Model

The project does not require a hardware backend, an LLVM backend, or an existing deep-learning runtime. For v1, a lightweight IR-walking simulator is proposed, capable of executing both the original (all-FP32) IR and the quantized IR produced by the transformation pass for the same input program, and comparing the two executions on the values that were actually quantized. This is sufficient to demonstrate the two properties that matter for this project: that the transformation pass produces IR that still executes correctly, and that a value labelled SAFE and converted to INT8 produces the same numeric result under simulation as its original FP32 counterpart, for every test input exercised.

4.13 Evaluation Strategy

The following metrics are proposed to evaluate the completed system once implemented; they are stated here as the planned evaluation strategy, not as results already obtained, since no implementation has yet been carried out:

- correctness of AST, TAC, and SSA construction against hand-verified test programs;
- correctness of the range and integrality analysis against manually computed expected ranges, including at least one program with a while loop to exercise the widening/termination logic;
- the number of values proposed as SAFE versus REJECTED, broken down by rejection reason (overflow, non-integral, unknown range);
- accuracy of reported source locations in diagnostics, verified against the actual source file;
- execution correctness of the quantized IR relative to the original FP32 IR under the simulator, for every SAFE value; and
- where meaningful, a simple count comparing FP32 versus INT8 storage for the values in a test program, to illustrate — without overstating — the potential memory benefit of the transformation.

No specific percentage improvement in speed, memory, or accuracy is claimed in advance of measurement; any such figures will be reported only once they have actually been measured against the implemented system.

4.14 End-to-End Example

Consider the small program

```
x = 10;
if (c) { x = 20; }
y = x + 5;
print y;
```

Parsing produces an AST with an assignment, an if statement, a second assignment, and a print statement, each node carrying its source line. Semantic analysis confirms x, y, and c are used consistently. Lowering to TAC and building the CFG yields the block structure of Figure 2, and converting to SSA form yields exactly the representation already shown in Figure 3 (x1 = 10; x2 = 20 on the branch; x3 = phi(x1,x2); y1 = x3 + 5). Range analysis then computes x1 → [10,10] (integral), x2 → [20,20] (integral), and, at the phi node, x3 → [10,20] (integral) — the same propagation already traced in Figure 4 — from which y1 → [15,25] (integral) follows. Applying the INT8 safety rule of Section 4.9 to y1: its range [15,25] lies within [-128,127], it is integral, and it derives only from supported operations, so it is labelled SAFE. The quantization pass therefore rewrites y1's defining instruction to an INT8 addition, and the diagnostic stage reports:

```
Line 3, Column 5
Variable: y1
Inferred range: [15, 25]
Integral: YES
INT8 Safety: SAFE
Reason: within INT8 bounds and integral on all paths
```

This single trace exercises every stage of the proposed pipeline — frontend, TAC, CFG, SSA, range/integrality analysis, safety decision, transformation, and diagnostics — on one small but representative program.

4.15 Implementation Strategy

Phase	Deliverable
Phase 1	Lexer, parser, and AST construction, including source-position tracking.
Phase 2	Semantic analysis and TAC generation.
Phase 3	Basic-block formation and CFG construction.
Phase 4	SSA construction (dominance frontiers, phi insertion, renaming).
Phase 5	Range and integrality analysis, including the widening/termination strategy for loops.
Phase 6	INT8 safety rule implementation.
Phase 7	Quantization transformation pass.
Phase 8	Lightweight simulator/backend for original and quantized IR.
Phase 9	Testing against hand-verified programs, evaluation per Section 4.13, and diagnostic-output refinement.

This roadmap is intentionally sequential: each phase depends on the IR produced by the previous one, and no phase requires functionality from a later phase to be tested in isolation. Detailed source code and implementation-level design (data structures, exact algorithmic pseudocode) are left out of this document by design, since this is a review document intended to establish the problem, objectives, background, and architecture ahead of implementation, not an implementation manual.

4.16 Current Implementation Status and Verification — Review 2

The Review 2 milestone establishes a working compiler core spanning the front end, intermediate representations, SSA-based analysis, conservative INT8 safety checking, selective quantization, diagnostics, and a lightweight execution model.

4.17 Implemented Compiler Stages

The current teaching-scale prototype successfully processes source code through the following pipeline:

Source Program → Lexer → Parser → AST → Semantic Analysis → TAC → Basic Blocks / CFG → SSA → Range + Integrality Analysis → INT8 Safety Analysis → Selective Quantization → Quantized SSA IR → Lightweight Simulator / Backend → Diagnostics / Evaluation.

Stage	Current Status	Evidence
Lexer	IMPLEMENTED	`src/lexer/`, lexer tests
Parser	IMPLEMENTED	`src/parser/`, parser tests
AST	IMPLEMENTED	`src/ast/`, AST tests
Semantic Analysis	IMPLEMENTED	`src/semantic/`, semantic tests
TAC Generation	IMPLEMENTED	`src/ir/`, TAC tests
CFG / Basic Blocks	IMPLEMENTED	`src/cfg/`, CFG tests
SSA Construction	IMPLEMENTED	`src/ssa/`, phi-node test and control-flow examples
Range Analysis	IMPLEMENTED	`src/analysis/`, interval propagation tests
Integrality Analysis	IMPLEMENTED	analysis tests and non-integral example
Loop Widening	IMPLEMENTED / CONSERVATIVE	loop analysis test and while-loop example
INT8 Safety Analysis	IMPLEMENTED	quantization tests and rejection examples
Quantization Transformation	IMPLEMENTED	quantization tests and safe arithmetic example
Backend / Simulator	IMPLEMENTED	simulator and integration tests
Diagnostics	IMPLEMENTED	diagnostic outputs with source locations
End-to-End Pipeline	IMPLEMENTED FOR CURRENT SCOPE	integration tests and example programs

4.18 Test and Verification Results

The verification suite confirms that unsafe or unproven values in the tested cases remain unquantized.

Test Category	Evidence	Result
Automated compiler test suite	`python -m unittest discover -s tests -v`	20/20 passed
Safe arithmetic	`examples/safe_arithmetic.qc`	PASS
Overflow rejection	`examples/overflow_rejection.qc`	PASS
Non-integral rejection	`examples/non_integral_rejection.qc`	PASS
If/else + phi	`examples/if_else_phi.qc`	PASS
While-loop + widening	`examples/while_loop.qc`	PASS
Original vs quantized if/else execution	simulator	[20] = [20]
Original vs quantized loop execution	simulator	[15] = [15]

The automated suite was run with the command and produced the output shown below.

```
python -m unittest discover -s tests -v

Ran 20 tests in 0.018s

OK
```

4.19 Representative End-to-End Compiler Outputs

1. Safe Arithmetic (`examples/safe_arithmetic.qc`)

Source:

```
int x = 10; int y = 20; int z = x + y; print z;
```

Analysis result:

```
x.1 -> [10,10] -> SAFE
y.1 -> [20,20] -> SAFE
z.1 -> [30,30] -> SAFE

3 analysed, 3 safe, 3 rewritten, 0 rejected.
```

Quantized IR:

```
x.1 : int8 = 10
y.1 : int8 = 20
z.1 : int8 = x.1 + y.1
```

2. Overflow Rejection (`examples/overflow_rejection.qc`)

Source:

```
int x = 100; int y = 30; int z = x + y; print z;
```

Analysis result:

```
x.1 -> [100,100] -> SAFE
y.1 -> [30,30] -> SAFE
z.1 -> [130,130] -> REJECTED
   Reason: possible overflow because 130 > 127

3 analysed, 2 safe, 2 rewritten, 1 rejected.
```

3. Non-integral Rejection (`examples/non_integral_rejection.qc`)

Source:

```
float x = 10.0; float y = 3.5; float z = x + y; print z;
```

Analysis result:

```
x.1 -> [10,10] -> integral -> SAFE
y.1 -> [3.5,3.5] -> non-integral -> REJECTED
z.1 -> [13.5,13.5] -> non-integral -> REJECTED

3 analysed, 1 safe, 1 rewritten, 2 rejected.
```

4. If/Else + SSA Phi (`examples/if_else_phi.qc`)

SSA phi node generated:

```
max.4 = phi(B3: max.3, B4: max.2)
```

Merged range:

```
max.4 -> [10,20]   (integral = YES)
```

The relational temporary `t1 = x.1 > y.1` is rejected and left unquantized because the relational comparison '>' is not supported by the current quantization transformation pass. Simulator execution of the original and the quantized IR yields identical output:

```
=== ORIGINAL SSA EXECUTION ===
[20]

=== QUANTIZED SSA EXECUTION ===
[20]
```

5. While Loop + Widening (`examples/while_loop.qc`)

Conservative loop widening may produce UNKNOWN when the analysis does not stabilize within the configured widening threshold. Such values are not quantized. Simulator execution of the original and the quantized IR yields identical output:

```
=== ORIGINAL SSA EXECUTION ===
[15]

=== QUANTIZED SSA EXECUTION ===
[15]
```

4.20 Limitations and Remaining Work

The implementation is a teaching-scale compiler prototype corresponding to the current project milestone, not a production compiler.

1. Loop widening is conservative and can produce UNKNOWN for loop-carried values when the range does not stabilize within the configured widening threshold.
2. Unsupported operations, such as the relational comparison demonstrated by '>', are not quantized.
3. The backend is a lightweight IR-walking simulator, not a real hardware backend or LLVM backend.
4. No empirical runtime, latency, energy, or hardware-power measurements have been performed yet.

4.21 Review 2 Milestone Summary

The Review 2 milestone establishes a working compiler core. The implemented numeric model evaluates INT8 eligibility using known ranges, bounds checking, integrality, supported operations, and safety violations. Safe integer-valued SSA operations can be lowered to int8 in the current implementation. Unsafe or unproven values remain unquantized.

5. Expected Outcomes

The proposed pipeline is expected to result in a compiler optimization pass that statically verifies, before a program is ever executed, whether an FP32 operation can be safely converted to INT8 — cutting the memory footprint of qualifying values by a factor of four and speeding up computation on edge hardware without introducing the runtime errors that an unverified precision reduction could cause. Concretely, the system is expected to deliver the following outcomes:

- **Static Safety Check:** verifies at compile time that a value remains an integer within [-128, 127] across every execution path, before any conversion is applied.
- **Selective Quantization:** rewrites only the operations certified safe into INT8, leaving every operation whose safety cannot be proven statically in its original FP32 form.
- **Traceable Output:** generates line-by-line diagnostics that report, for every variable, whether it was converted or rejected and the specific reason for that decision.

Together, these outcomes keep the safety decision and the transformation itself fully separate and auditable, so every quantization choice the compiler makes can be traced back to the exact source line responsible for it.

Syllabus Mapping

The table below connects the project's components to the relevant modules of the BCSE307L Compiler Design syllabus. Only genuinely relevant concepts are listed.

BCSE307L Concept	Project Component
Lexical Analysis / Tokens (Module 1)	Lexer and token stream with source-position tracking
Syntax Analysis / Parsing (Module 2)	Parser and AST construction
Intermediate Code Generation / TAC (Module 4)	TAC generation from the AST
Control Flow / Basic Blocks (Module 5)	Basic-block formation and CFG construction
Data-Flow Analysis (Module 5)	Static range and integrality analysis over the CFG
Machine-Independent Optimization (Module 5)	INT8 safety decision and quantization transformation pass
Static Single Assignment (Module 7)	SSA construction with phi nodes as the analysis representation
Code Generation / Virtual Machine Code (Module 6)	Lightweight simulator / backend executing original and quantized IR

References

[1] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, “Edge intelligence: Paving the last mile of artificial intelligence with edge computing,” Proc. IEEE, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.

[2] J. Chen and X. Ran, “Deep learning with edge computing: A review,” Proc. IEEE, vol. 107, no. 8, pp. 1655–1674, Aug. 2019.

[3] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713.

[4] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” arXiv:1806.08342, 2018.

[5] P. Cousot and R. Cousot, “Static determination of dynamic properties of programs,” in Proc. 2nd Int. Symp. on Programming (ISOP), Paris, France: Dunod, 1976, pp. 106–130.

[6] P. Cousot and R. Cousot, “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints,” in Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL), 1977, pp. 238–252.

[7] F. Bourdoncle, “Efficient chaotic iteration strategies with widenings,” in Formal Methods in Programming and Their Applications, LNCS vol. 735. Berlin, Germany: Springer, 1993, pp. 128–141.

[8] B. Blanchet, P. Cousot, R. Cousot, J. Feret, L. Mauborgne, A. Miné, D. Monniaux, and X. Rival, “A static analyzer for large safety-critical software,” in Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI), 2003, pp. 196–207.

[9] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, “Efficiently computing static single assignment form and the control dependence graph,” ACM Trans. Program. Lang. Syst., vol. 13, no. 4, pp. 451–490, Oct. 1991.

[10] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis and transformation,” in Proc. Int. Symp. on Code Generation and Optimization (CGO), 2004, pp. 75–86.

[11] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, “Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines,” in Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI), 2013, pp. 519–530.

[12] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, “TVM: An automated end-to-end optimizing compiler for deep learning,” in Proc. 13th USENIX Symp. on Operating Systems Design and Implementation (OSDI), 2018, pp. 578–594.